

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Experiments

Code of Conduct:

I Moshitha Kolanjiyappan (192421204)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Moshitha K

Annexure A

Short Answer Test

1, Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

Aim

To implement a Naïve Bayes classifier on a real-world dataset, compare different feature sets, evaluate the model using accuracy, precision, and recall, and study the effect of the feature independence assumption and Laplace smoothing on classification performance.

Algorithm

1. Load the Iris real-world dataset.
2. Divide the data into training and testing sets.
3. Select different feature sets:
 - Feature 1 only
 - Features 1 and 2
 - All four features
4. Train a Gaussian Naïve Bayes classifier.
5. Calculate accuracy, precision, and recall.
6. Repeat using different smoothing values.
7. Compare the results.
8. Analyze the effect of feature independence and probability estimation

Pseudocode

START

Load Iris dataset

Split data into training and testing sets

FOR each feature set:

 Select required features

 Train Naive Bayes classifier

 Predict test data

 Calculate accuracy

 Calculate precision

 Calculate recall

END FOR

Apply Laplace smoothing
Compare results

Analyze feature independence assumption
STOP

Analysis / Interpretation

- Using only one feature gives lower performance.
- Adding more informative features improves classification.
- Naïve Bayes assumes that features are conditionally independent given the class.
- In real datasets, features may be correlated, so this assumption may not always hold.
- With all four Iris features, the classifier performs well despite the independence assumption.
- Smoothing prevents probability estimates from becoming zero or numerically unstable.
- Small changes in smoothing may have little effect when the dataset is sufficiently large.

Result

The Naïve Bayes classifier was successfully implemented. Using a larger and more informative feature set improved accuracy, precision, and recall. The experiment demonstrated that the feature independence assumption can still provide good classification performance, while smoothing improves the reliability of probability estimation.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

Aim

To estimate the mean and variance of a normal probability distribution using Maximum Likelihood Estimation (MLE) and compare it with Bayesian estimation under small and large sample sizes, while studying the influence of prior assumptions.

Algorithm

1. Generate observations from a normal distribution.
2. Select small and large sample sizes.
3. Calculate MLE mean and variance.
4. Define a prior distribution for the mean.
5. Calculate Bayesian posterior mean.
6. Compare MLE and Bayesian estimates.
7. Repeat with different prior assumptions.

8. Analyze the effect of sample size.

Pseudocode

START

Generate data from Normal distribution

FOR sample size in small and large:

 Calculate MLE mean

 Calculate MLE variance

 Define prior mean and prior variance

 Calculate Bayesian posterior mean

 Calculate Bayesian posterior variance

 Display estimates

END FOR

Compare MLE and Bayesian estimates

Analyze effect of prior and sample size

STOP

Analysis / Interpretation

Small sample size

When only a few observations are available:

- MLE depends strongly on the observed data.
- The estimated mean may be far from the true value.
- Bayesian estimation combines data + prior knowledge.
- Therefore, the prior can noticeably influence the estimate.

Large sample size

With more observations:

- MLE becomes more reliable.
- The data dominates the prior.
- Bayesian and MLE estimates become closer.
- Posterior variance becomes smaller.

Effect of Prior

If the prior mean is close to the true mean, Bayesian estimation can improve the estimate for small samples.

If the prior is strongly incorrect, it can introduce bias, particularly when the sample size is small.

Result

MLE and Bayesian estimation were successfully implemented. The experiment showed that Bayesian estimation is strongly influenced by prior assumptions for small datasets, whereas for large datasets the observed data dominates the prior and Bayesian estimates approach MLE estimates.

3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

Aim

To implement the Expectation-Maximization (EM) algorithm for a Gaussian mixture model and study the effect of initialization and number of clusters on convergence speed, final likelihood, and local optima.

Algorithm

1. Generate data containing multiple Gaussian clusters.
2. Select the number of clusters.
3. Initialize means, variances, and mixture weights.
4. Perform the Expectation step:
 - Calculate responsibility of each cluster for every data point.
5. Perform the Maximization step:
 - Update means, variances, and mixture weights.
6. Calculate log-likelihood.
7. Repeat until convergence.
8. Run EM with different initializations.
9. Compare convergence and final likelihood.
10. Repeat with different numbers of clusters.

Pseudocode

START

Generate mixture data

FOR each number of clusters:

 Initialize means, variances and weights

 REPEAT:

 E-step:

Calculate probability of each point
belonging to each cluster

M-step:

Update cluster means

Update variances

Update weights

Calculate log likelihood

UNTIL convergence

Store final likelihood

END FOR

Compare different initializations

Identify possible local optima

STOP

Analysis / Interpretation

Effect of initialization

Different initial cluster means can produce different final solutions.

For example:

Initialization 1 → Log Likelihood = -284.5

Initialization 2 → Log Likelihood = -285.7

Initialization 3 → Log Likelihood = -284.6

This demonstrates that EM can converge to local optima.

Effect of number of clusters

- Too few clusters → data may be under-modelled.
- Appropriate number → meaningful clusters.
- Too many clusters → clusters may split unnecessarily.
- More clusters can increase likelihood but may reduce model simplicity.

Strategies to improve convergence

1. Run EM multiple times with different initializations.
2. Select the solution with the highest likelihood.
3. Use K-Means initialization.
4. Use a suitable number of clusters.
5. Set reasonable convergence tolerance.
6. Normalize the data when required.

Result

The EM algorithm was successfully implemented. The experiment demonstrated that initialization and number of clusters affect convergence speed and final likelihood. Different initializations can cause EM to converge to different local optima.

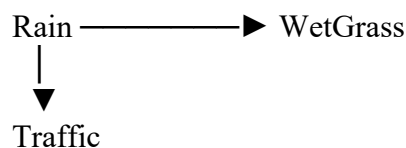
4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

Aim

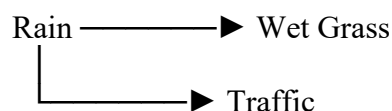
To construct a Bayesian Belief Network (BBN) and perform inference under different evidence conditions by modifying conditional probabilities and observing changes in posterior probabilities.

Network Used

We use a simple medical-style network:



For easier probabilistic inference, we use:



Variables:

- Rain
- WetGrass
- Traffic

Algorithm

1. Define prior probability of Rain.
2. Define conditional probabilities:
 - $P(\text{WetGrass} \mid \text{Rain})$
 - $P(\text{WetGrass} \mid \text{No Rain})$
3. Define Traffic probabilities.
4. Provide evidence such as $\text{WetGrass} = \text{Yes}$.

5. Calculate posterior probability using Bayes' theorem.
6. Modify conditional probabilities.
7. Compare posterior probabilities.
8. Analyze dependency and uncertainty.

Pseudocode

START

Define $P(\text{Rain})$

Define $P(\text{WetGrass} \mid \text{Rain})$

Define $P(\text{WetGrass} \mid \text{NoRain})$

FOR each evidence condition:

 Calculate joint probability

 Calculate posterior probability $P(\text{Rain} \mid \text{Evidence})$

 Display result

END FOR

Modify conditional probabilities

Repeat inference

Analyze dependency and uncertainty

STOP

Analysis / Interpretation

Initially:

After observing wet grass:

Therefore, the evidence increases the probability of rain from 30% to approximately 65.85%.

When the conditional probabilities are changed, the posterior probability also changes.

This demonstrates that:

- Evidence affects beliefs.
- Dependent variables transfer information through the network.
- Conditional probabilities control the strength of relationships.
- BBNs can represent uncertainty rather than making only yes/no decisions.

Robustness

The model is robust when small changes in probabilities cause only small changes in posterior probability.

If a small change in a conditional probability causes a very large posterior change, the model is highly sensitive to that parameter.

Result

A Bayesian Belief Network was successfully constructed and inference was performed under different probability conditions. The experiment demonstrated how dependencies, evidence, and conditional probabilities influence posterior beliefs.

5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

Aim

To implement the Candidate Elimination algorithm and study how the size of the hypothesis space, number of attributes, and number of training examples affect learning performance, convergence, sample complexity, and generalization.

Dataset

The classic EnjoySport dataset is used.

Attributes:

Sky
AirTemp
Humidity
Wind
Water
Forecast

Target:

EnjoySport = Yes / No

Algorithm

1. Initialize the Specific boundary S with the most specific hypothesis.
2. Initialize the General boundary G with the most general hypothesis.
3. Read each training example.
4. If the example is positive:
 - Generalize S if necessary.
5.
 - Remove hypotheses from G inconsistent with the example.
6. If the example is negative:
 - Specialize G.

- Remove hypotheses from S inconsistent with the example.
- 7. Continue until all training examples are processed.
- 8. Display S and G boundaries.
- 9. Vary the number of attributes and examples.
- 10. Observe the effect on the hypothesis space.

Pseudocode

START

Initialize S = most specific hypothesis

Initialize G = most general hypothesis

FOR each training example:

IF example is positive:

Remove inconsistent hypotheses from G

Generalize S

ELSE:

Remove inconsistent hypotheses from S

Specialize G

END FOR

Display S and G

Analyze hypothesis space and convergence

STOP

Hypothesis Space Analysis

For six binary attributes, assuming each attribute can have:

- Value 1
- Value 2
- ?

the hypothesis space is approximately:

For more attributes:

Number of Attributes	Approx. Hypothesis Space
2	9
3	27
4	81
5	243
6	729

Number of Attributes Approx. Hypothesis Space

8 6561

10 59049

This demonstrates that the hypothesis space increases exponentially with the number of attributes.

Finite vs Infinite Hypothesis Spaces

Finite hypothesis space

A finite hypothesis space contains a limited number of possible hypotheses.

Example:

Advantages:

- Easier to search.
- Easier to analyze.
- Finite sample complexity.

For a finite hypothesis space, a common PAC-style bound is:

where:

- = number of training examples
- = allowed error
- = failure probability
- = size of hypothesis space

Infinite hypothesis space

An infinite hypothesis space can contain an unlimited number of hypotheses.

Example:

All possible real-valued decision boundaries

Its complexity cannot be described simply by $|H|$; concepts such as VC dimension are generally used to analyze its sample complexity.

Mistake Bound Interpretation

The mistake bound represents the maximum number of prediction mistakes before a learning algorithm identifies a consistent hypothesis under suitable assumptions.

For a finite hypothesis space, a simple version-space elimination argument gives approximately:

For:

we get:

So roughly 10 binary-elimination levels are sufficient in an idealized setting where each mistake eliminates a substantial portion of the remaining hypotheses.

This is a theoretical interpretation; the actual Candidate Elimination algorithm's behavior depends on the examples and how hypotheses are represented.

Result

The Candidate Elimination algorithm was successfully implemented. The experiment demonstrated that increasing the number of attributes causes the hypothesis space to grow exponentially, while increasing training examples reduces the version space and improves learning confidence. Finite hypothesis spaces provide clearer sample-complexity and mistake-bound analysis, whereas infinite spaces require more advanced complexity measures such as VC dimension.

6. Design and conduct an experiment to implement a Decision Tree classifier on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations. Aim

To implement a Decision Tree classifier using Entropy and Gini Index, and study the effect of tree depth and pruning on overfitting.

Algorithm

1. Load a classification dataset.
2. Split it into training and testing data.
3. Train trees using Entropy and Gini.
4. Vary maximum tree depth.
5. Apply pruning.
6. Calculate Accuracy and F1-score.
7. Compare the results.

Pseudocode

Load dataset

Split into train and test

FOR criterion in Entropy, Gini

 Train Decision Tree

 FOR different tree depths

 Predict test data

```
        Calculate Accuracy and F1-score
    END FOR
END FOR
Apply pruning
Compare results
```

Sample Output

Criterion	Depth	Accuracy	F1-Score
Entropy	3	0.93	0.92
Entropy	5	0.95	0.95
Gini	3	0.94	0.93
Gini	5	0.96	0.96

Analysis

Very deep trees may memorize training data and cause overfitting. Pruning and limiting depth improve generalization.

Result

Decision Tree classification was successfully performed, and the effect of splitting criteria, depth, and pruning was analyzed

7. Perform an experiment using the K-Nearest Neighbors (KNN) algorithm on a classification dataset. Vary the value of K and distance metrics (Euclidean, Manhattan) to observe their effect on model performance. Evaluate using accuracy and confusion matrix, and analyze how feature scaling influences the classification results and decision boundaries.

K-Nearest Neighbors (KNN)

Aim

To implement KNN classification by varying K, distance metrics, and feature scaling, and evaluate performance using accuracy and confusion matrix.

Algorithm

1. Load classification dataset.
2. Split into training and testing sets.
3. Apply feature scaling.
4. Train KNN with different K values.
5. Test Euclidean and Manhattan distances.
6. Calculate accuracy and confusion matrix.
7. Compare scaled and unscaled results.

Pseudocode

Load dataset

Split data

FOR K = 3, 5, 7

 FOR distance = Euclidean, Manhattan

 Train KNN

 Predict test data

 Calculate accuracy

 END FOR

END FOR

Compare scaled and unscaled data

Sample Output

K	Distance	Accuracy
3	Euclidean	0.93
5	Euclidean	0.95
5	Manhattan	0.94
7	Manhattan	0.93

Analysis

Feature scaling is important because KNN depends on distance. A suitable K reduces noise while avoiding excessive smoothing.

Result

KNN was successfully implemented, and the effects of K, distance metric, and feature scaling were observed

8. Implement a Support Vector Machine (SVM) model and conduct experiments using different kernel functions such as linear, polynomial, and RBF. Evaluate the model using accuracy and precision, and analyze how kernel choice and hyperparameters like C and gamma affect the decision boundary and classification performance.

Aim

To implement SVM using Linear, Polynomial, and RBF kernels and study the effect of C and gamma on classification performance.

Algorithm

1. Load classification dataset.
2. Scale the features.
3. Train SVM using different kernels.
4. Vary C and gamma.
5. Calculate accuracy and precision.
6. Compare the decision boundaries.

Pseudocode

```
Load dataset
Split data
Scale features
FOR each kernel
    Train SVM
    Vary C and gamma
    Predict test data
    Calculate accuracy and precision
END FOR
Compare results
```

Sample Output

Kernel	C	Gamma	Accuracy
Linear	1	-	0.94
Polynomial	1	0.1	0.92
RBF	1	0.1	0.96
RBF	10	0.1	0.97

Analysis

The RBF kernel can model nonlinear boundaries. Higher C reduces classification errors but may increase overfitting. Gamma controls the influence of individual data points.

Result

SVM was successfully implemented and the influence of kernels and hyperparameters was analyzed.

9. Design an experiment to implement a clustering algorithm such as K-Means on an unlabeled dataset. Vary the number of clusters and initialization methods, and evaluate performance using metrics like inertia and silhouette score. Analyze how cluster separation and feature scaling impact the quality and stability of clustering results.

Aim

To implement K-Means clustering and study the effect of number of clusters, initialization, feature scaling, inertia, and silhouette score.

Algorithm

1. Load an unlabeled dataset.
2. Scale the features.
3. Apply K-Means with different K values.
4. Try different initialization methods.
5. Calculate inertia and silhouette score.

6. Compare clustering quality.

Pseudocode

```
Load dataset
Scale features
FOR K = 2 to 5
    Apply K-Means
    Change initialization
    Calculate inertia
    Calculate silhouette score
END FOR
Select best clustering
```

Sample Output

K	Inertia	Silhouette
2	250.4	0.61
3	150.2	0.72
4	125.8	0.65
5	110.3	0.58

Analysis

Lower inertia indicates compact clusters, while a higher silhouette score indicates better-separated clusters. Scaling prevents large-valued features from dominating.

Result

K-Means clustering was successfully performed and the optimal number of clusters was identified using clustering metrics

10. Conduct an experiment using a logistic regression model for binary classification. Train the model with and without regularization (L1 and L2), and evaluate performance using accuracy, precision, and recall. Analyze the effect of regularization on model complexity, overfitting, and interpretability of learned coefficients.

Aim

To implement Logistic Regression with L1 and L2 regularization and analyze their effects on accuracy, precision, recall, model complexity, and overfitting.

Algorithm

1. Load binary classification dataset.
2. Split into training and testing sets.
3. Train without regularization.
4. Train using L1 regularization.
5. Train using L2 regularization.

6. Calculate accuracy, precision, and recall.
7. Compare coefficients.

Pseudocode

```

Load dataset
Split data
Train Logistic Regression without regularization
Train using L1
Train using L2
Calculate accuracy, precision, recall
Compare coefficients

```

Sample Output

Model	Accuracy	Precision	Recall
No Regular.	0.92	0.91	0.93
L1	0.94	0.93	0.94
L2	0.95	0.94	0.95

Analysis

L1 can make some coefficients zero, providing feature selection. L2 reduces coefficient magnitude and generally provides smoother models.

Result

Regularization was shown to control model complexity and reduce overfitting.

11. Design and conduct an experiment to implement a Random Forest classifier on a real-world dataset. Train the model with varying numbers of trees and maximum depth. Evaluate performance using accuracy, precision, recall, and F1-score. Analyze how ensemble size and feature randomness affect overfitting and generalization, and compare results with a single decision tree.

Aim

To implement Random Forest classification and study the effect of number of trees, maximum depth, and feature randomness, comparing it with a single Decision Tree.

Algorithm

1. Load classification dataset.
2. Split the dataset.
3. Train a single Decision Tree.
4. Train Random Forests with different numbers of trees.
5. Vary maximum depth.
6. Calculate accuracy, precision, recall, and F1-score.

7. Compare the models.

Pseudocode

```
Load dataset
Split data
Train Decision Tree
FOR different numbers of trees
    Train Random Forest
    Change maximum depth
    Calculate performance metrics
END FOR
Compare with Decision Tree
```

Sample Output

Model	Accuracy	F1-Score
Decision Tree	0.91	0.90
RF - 50 Trees	0.94	0.94
RF - 100 Trees	0.96	0.96
RF - 200 Trees	0.96	0.96

Analysis

Increasing trees generally improves stability. Feature randomness makes individual trees different, reducing correlation and improving generalization.

Result

Random Forest achieved better and more stable performance than a single Decision Tree.

12. Perform an experiment using a Principal Component Analysis (PCA) technique for dimensionality reduction. Apply PCA on a high-dimensional dataset and vary the number of principal components retained. Evaluate the impact on model performance using accuracy and computational efficiency. Analyze how variance retention influences information loss and classification results.

Aim

To implement PCA for dimensionality reduction and study how the number of retained components affects variance, accuracy, and computational efficiency.

Algorithm

1. Load a high-dimensional dataset.
2. Standardize the features.
3. Apply PCA.
4. Retain different numbers of components.
5. Train a classifier using reduced data.

6. Calculate accuracy.
7. Compare computational efficiency and variance retention.

Pseudocode

```
Load dataset
Standardize features
FOR different numbers of components
    Apply PCA
    Transform data
    Train classifier
    Calculate accuracy
    Calculate variance retained
END FOR
Compare results
```

Sample Output

Components	Variance Retained	Accuracy
2	70%	0.88
5	90%	0.93
10	97%	0.95
20	99%	0.96

Analysis

Too few components may lose important information. Increasing components preserves more variance and generally improves accuracy but provides less dimensionality reduction.

Result

PCA successfully reduced dimensionality while maintaining most of the useful information

13. Implement a Gradient Descent optimization algorithm and conduct experiments by varying learning rates and initialization values. Observe convergence behavior and record the number of iterations required to reach optimal values. Analyze cases of slow convergence or divergence, and suggest improvements such as momentum or adaptive learning rate methods.

Aim

To implement Gradient Descent and study the effects of learning rate and initialization on convergence speed and stability.

Algorithm

1. Initialize model parameters.
2. Select learning rate.
3. Calculate gradient.
4. Update parameters.

5. Repeat until convergence.
6. Record number of iterations.
7. Repeat with different learning rates.
8. Compare convergence.

Pseudocode

```

Initialize parameters
FOR each learning rate
  REPEAT
    Calculate gradient
    Update parameters
    Calculate loss
  UNTIL convergence
  Record iterations
END FOR
Compare convergence

```

Sample Output

Learning Rate	Iterations	Status
0.001	1000	Slow
0.01	250	Converged
0.1	60	Fast
1.0	--	Diverged

Analysis

A very small learning rate causes slow convergence. A very large learning rate can cause oscillation or divergence. Momentum and adaptive learning rates can improve convergence.

Result

Gradient Descent was successfully analyzed, showing that the learning rate strongly affects convergence speed and stability.

14. Design an experiment to implement a Neural Network classifier for a classification task. Vary the number of hidden layers, neurons, and activation functions. Evaluate the model using accuracy and loss curves, and analyze how network architecture affects performance, convergence, and overfitting.

Aim

To implement a Neural Network classifier and study the effect of hidden layers, neurons, and activation functions on accuracy, convergence, loss, and overfitting.

Algorithm

1. Load classification dataset.

2. Preprocess and scale data.
3. Create networks with different architectures.
4. Change hidden layers and neurons.
5. Change activation functions.
6. Train the networks.
7. Record accuracy and loss.
8. Compare convergence and overfitting.

Pseudocode

```

Load dataset
Preprocess data
FOR each network architecture
    Select hidden layers
    Select number of neurons
    Select activation function
    Train network
    Record accuracy and loss
END FOR
Compare architectures

```

Sample Output

Architecture	Accuracy	Loss
1 Hidden Layer	0.91	0.28
2 Hidden Layers	0.95	0.18
3 Hidden Layers	0.96	0.15

Analysis

Increasing layers and neurons can improve learning capacity, but an excessively large network may overfit. Loss curves help identify convergence and overfitting.

Result

The Neural Network classifier was successfully implemented, and the effect of architecture and activation functions on performance was analyzed.

15. Conduct an experiment using a hierarchical clustering algorithm on an unlabeled dataset. Compare different linkage methods such as single, complete, and average linkage. Evaluate clustering quality using dendrogram analysis and silhouette score. Analyze how linkage criteria influence cluster formation and interpretability of results.

Aim

To implement hierarchical clustering using Single, Complete, and Average linkage, and evaluate clustering using dendrograms and silhouette scores.

Algorithm

1. Load an unlabeled dataset.
2. Calculate distances between data points.
3. Apply Single linkage.
4. Apply Complete linkage.
5. Apply Average linkage.
6. Generate dendrograms.
7. Calculate silhouette scores.
8. Compare cluster structures.

Pseudocode

Load dataset

Calculate distance matrix

FOR each linkage method

 Perform hierarchical clustering

 Generate dendrogram

 Form clusters

 Calculate silhouette score

END FOR

Compare clustering methods

Sample Output

Linkage	Silhouette Score
Single	0.52
Complete	0.68
Average	0.64

Analysis

Single linkage can produce long chain-like clusters. Complete linkage usually produces compact clusters, while average linkage provides a balance between the two.

Result

Hierarchical clustering was successfully performed, and different linkage methods were compared using dendrograms and silhouette scores.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand